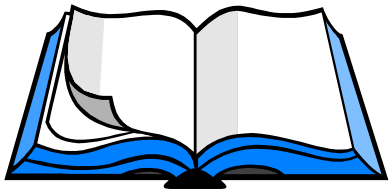
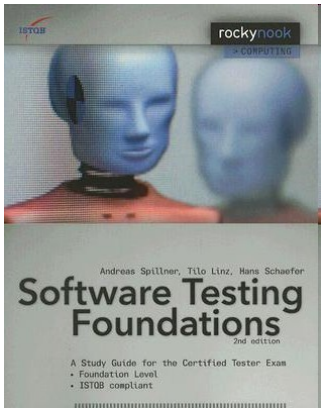
Software Testing

Module review

Recap: Basic Reading List



Andreas Spillner, Tilo Linz, Hans Schaefer, *Software Testing Foundations*, 2nd Edition, RockyNook Books



Dorothy Graham, Isabel Evans, Erik Van Veenendaal and Rex Black, *Foundations of Software Testing*

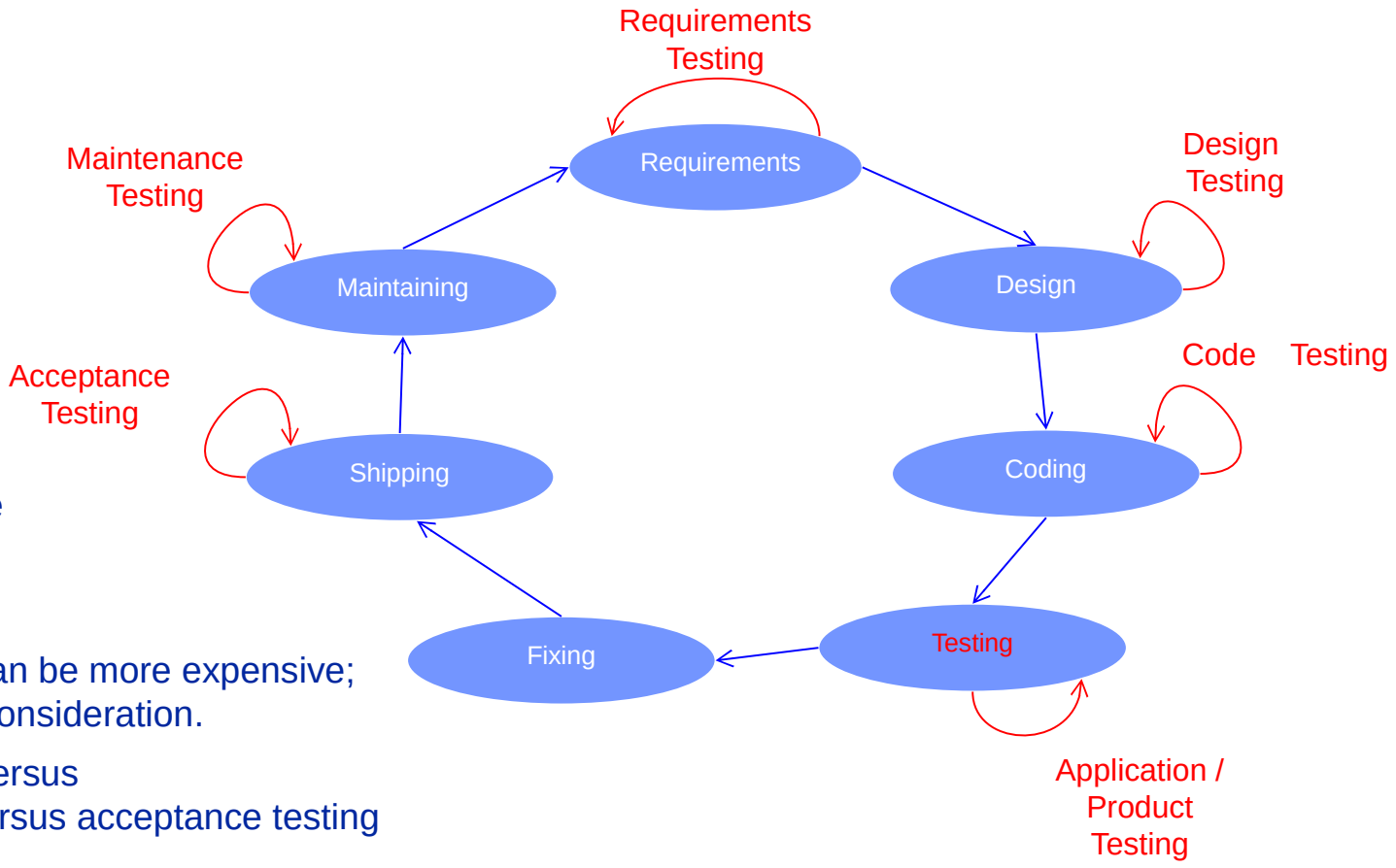
Various other links to papers and articles have been provided throughout the term

General Background

- Where does testing fit in?
- Why is testing important?
- System testing is one of the final activities.
- Terminology
- How much testing is enough?
- Testing strategy

How does testing fit in?

Development lifecycle: Broader scope of testing domain (**testing throughout the lifecycle**)



- Testing is everywhere

- Testing is expensive

- Not testing enough can be more expensive; there is an economic consideration.

- Note on integration versus system/qualification versus acceptance testing

- This module is concerned with all of these different types of testing.

Why is testing important?

- We reviewed a number of examples of projects where an absence of testing / quality assurance resulted in major failures.
- We need to reduce the risk and potential impact of failures.

System testing is one of the final activities.

Because system testing is one of the last tasks prior to customer delivery, well intentioned and appropriately planned system testing activities are often subject to significant time pressure.

Functional and non-functional considerations….

In this instance – it is not good to be last.

Causes of software defects

A human being can make an error (mistake), which produces a defect (fault, bug) in the code, in software or a system, or in a document

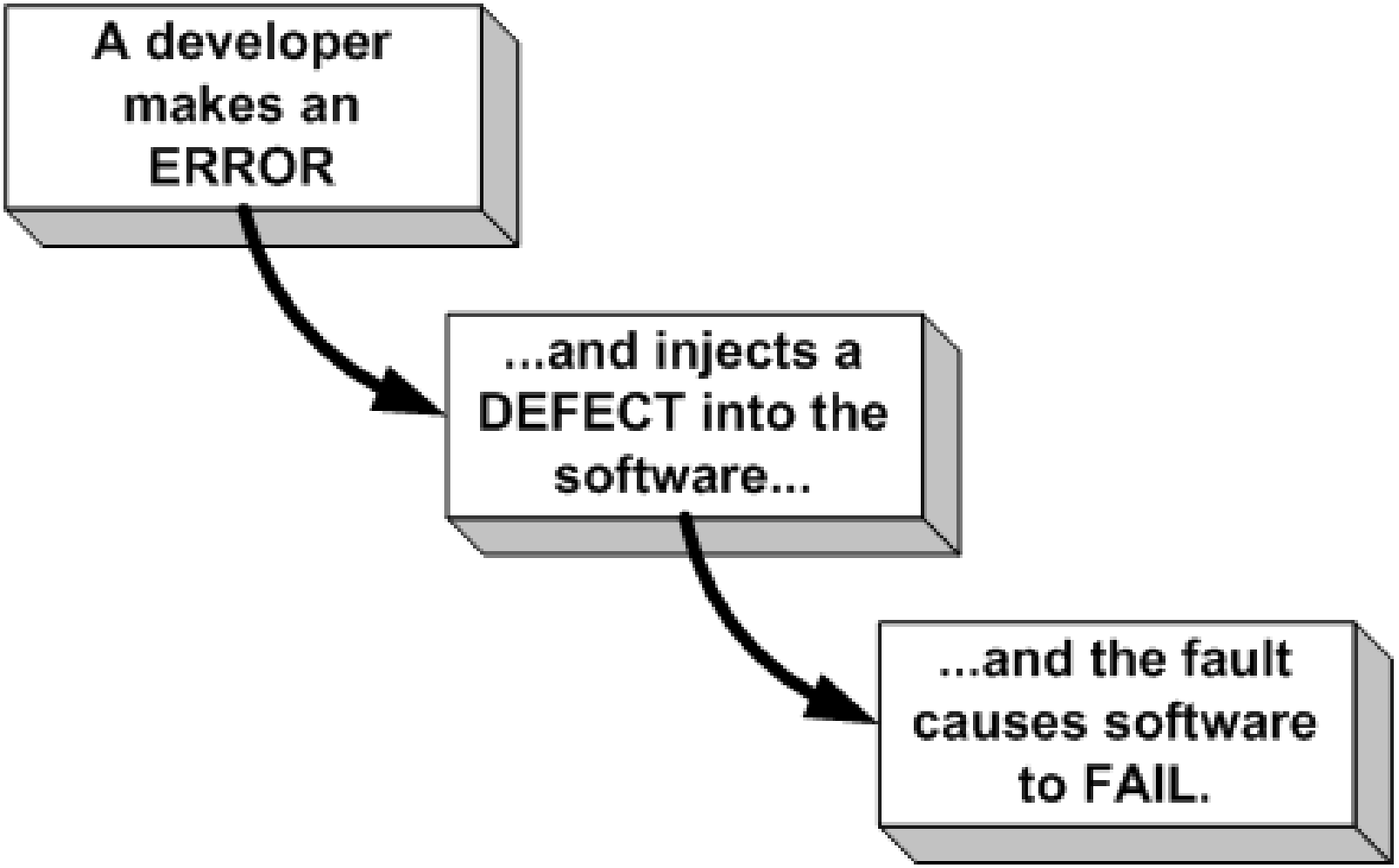
If the code is executed, the system will fail, causing a failure

Defects in software, systems or documents may result in failures, but not all defects do so

Failures can be caused by environmental conditions as well:

- radiation, magnetism, electronic fields, and pollution can cause faults in firmware or influence the execution of software by changing hardware conditions.

Errors, defects and failures



How much testing is enough?

A perplexing question

* There's no upper limit

If we don't test enough, we get blamed for faults in production

The tester “can't win”.

** Need to measure, use metrics.

Think of managing a deficit or a surplus.

Software Testing

- Testing != Debugging (discuss IDEs)
- Definition of testing / a test
- Static versus dynamic testing
- Importance of baselines
- Principles of testing
- The test process
- Psychology of testing
- Degrees of independence

What is testing?

Testing

The systematic exploration of a component/system with the main aim of finding and reporting defects.

Testing rigorously examines the component/system behaviour and reports defects found.

Tests are repeated to ensure that defect corrections have been effective.

Debugging

A process undertaken by developers

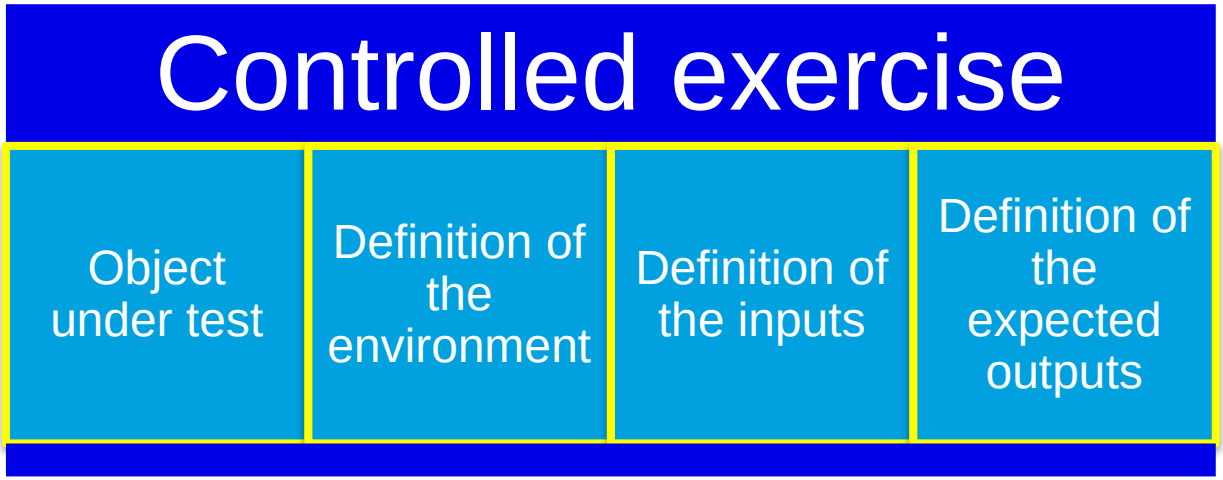
to identify the cause of defects in code and undertake corrections.

Debugging is done first to ensure that the component or system is at a level to enable rigorous testing.

Debugging can be used to understand the root cause of observed failures.

What is testing?

- What is a Test?



- Thus once a Test is run, you'll get
  - An actual result / output
  - A decision on its success

Static and Dynamic Testing

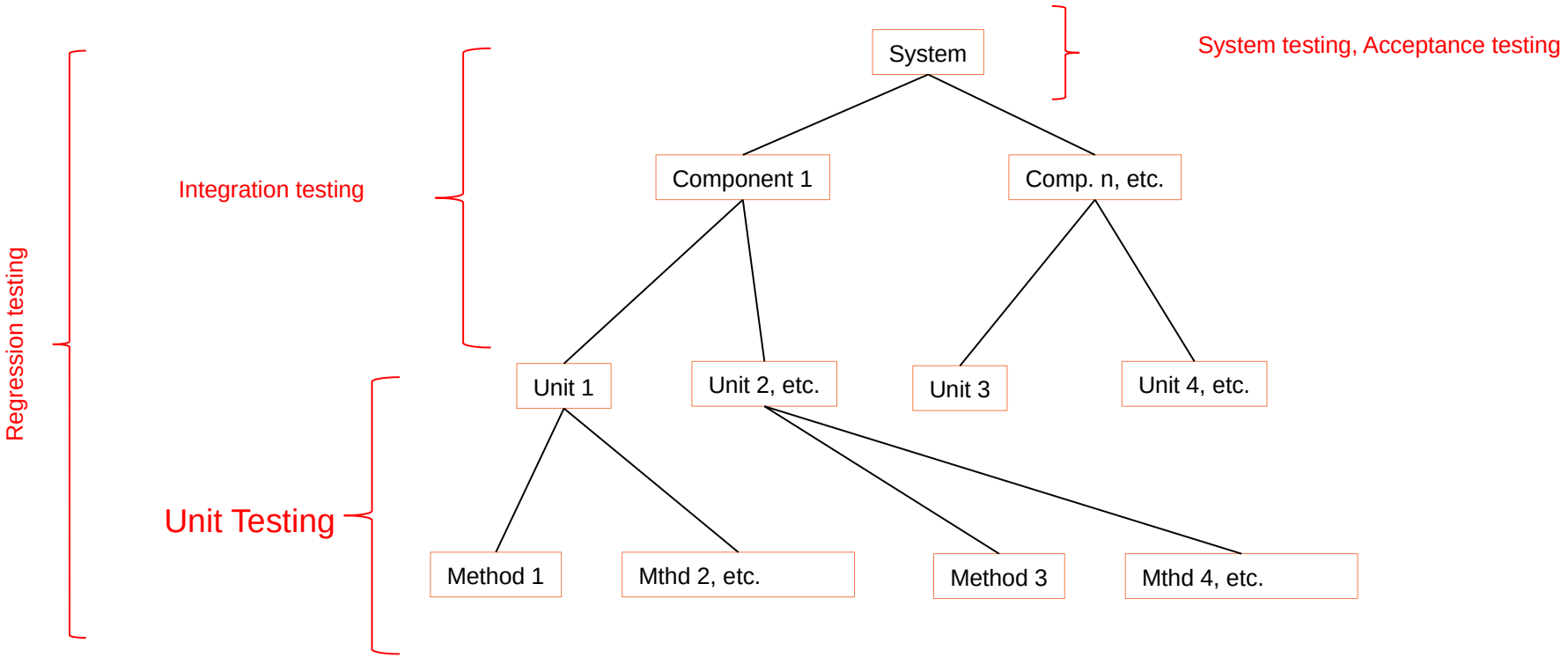
Static Testing

- Testing without executing the program
  - This include software inspections and some forms of analysis
  - Very effective at finding certain kinds of problems – especially “potential” faults, that is, problems that could lead to faults when the program is modified

Dynamic Testing

- Testing by executing the program with real inputs
  - Includes various techniques but requires the coding stage to be completed.

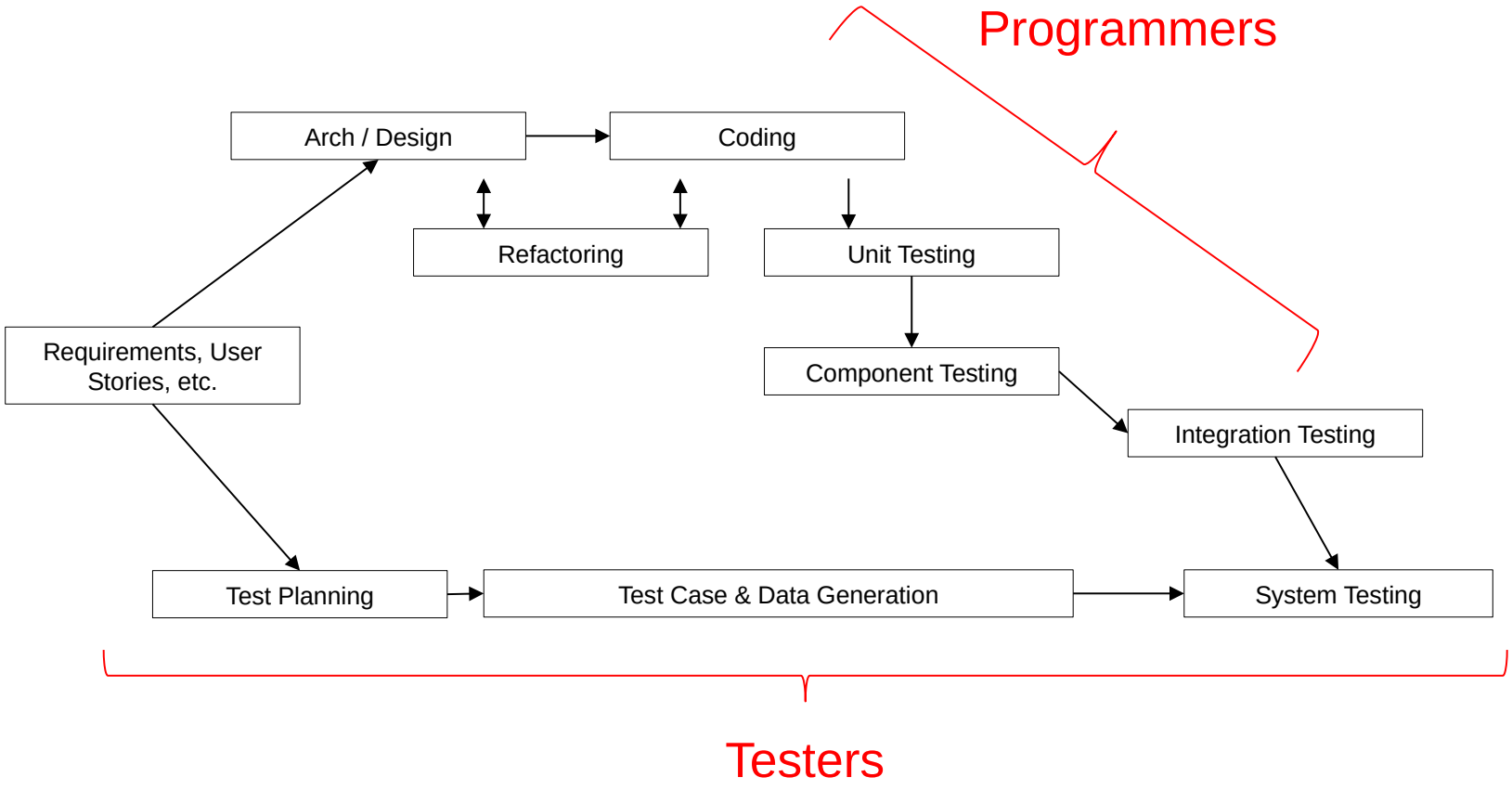
Types of Dynamic Testing



Some principles of testing

1. Testing shows the presence of defects but not their absence
2. Exhaustive testing is impossible
3. Testing activities should start as early as possible
4. Defect clustering
5. Pesticide paradox
6. Testing is context dependent
7. Absence-of-errors fallacy (requirements may have been incomplete or inaccurate!)

Testing Sequencing



Baseline as an oracle[1] for required behaviour

When we test we get an actual result

We compare results with requirements to determine whether a test has passed

A baseline document describes how we require the system to behave

User requirement, design, spec. etc.

Sometimes the 'old system' tells us what to expect.

[1] *A source to determine expected results to compare with the actual result of the software under test. An oracle may be the existing system (for a benchmark), a user manual, or an individual's specialized knowledge, but should not be the code.*

Expected results

- If we don't define expected result before we execute the test...
  - a plausible, but erroneous, result may be interpreted as the correct result
  - there may be a subconscious desire to see the software pass the test
- Expected results must be defined before test execution, derived from a baseline.

Exit Criteria

Trigger to say: "we've done enough"

Objective, non-technical for managers

Measurable, achievable target. Some typical types of criterion which are used regularly are listed below:

- 80% coverage achieved
- all tests executed without failure
- all outstanding incidents waived
- all critical business scenarios covered.

The Test Process

The fundamental test process consists of the following main activities:

- test planning
- test control
- test analysis
- test design
- test implementation (aka preparation)
- test execution and recording
- evaluating exit criteria and reporting
- test closure activities

Psychology of testing

A tester:

- Employs professional cynicism.
- Tries to prove that the solution does not work (they are detached from the ‘ownership’ and inherent protection Developers have for their code).

A Developer’s job is to prove the solution does work.

Testers see other and different defects to the author/developer.

Testers are unbiased.

Testers do not make assumptions.

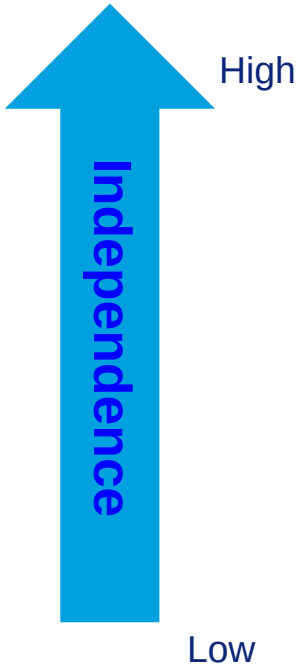
Degrees of independence

Outsourced test team
(e.g. contractors, testing consultancy)

Specialist testers such as usability testers,
performance testers, security testers etc.

Separate permanent test team

Developers used as testers



Lifecycle Models

- Software development process
- Software lifecycle models

Life-Cycle Models – And impact on testing…

- Build-and-fix model
- Waterfall model
- Rapid prototyping model
- Spiral model
- Prototyping Model
- Phased Development Model
  - incremental development model
  - iterative development model
- Formal Systems Development
- Agile models (model or method)
  - Extreme programming
- Continuous Software Engineering

See – Ian Sommerville, Software Engineering for details on process models

Analysis & Design Techniques

- Data flow analysis
- Program flowcharts
- Test design techniques
- Black box
- White box
- Equivalence partitioning
- Boundary value analysis

Data Flow Analysis

How data is used on the different paths through the code is checked.
There are **3 different usage states for** each data **variable**.

Undefined (u) The data has no defined value.

Defined (d) The data is assigned a value.

Referenced (r) The data is used.

Data flow analysis cannot detect errors but it **can identify Anomalies** - These anomalies are usually high risk features.

ur-anomaly - An undefined data item is read on a program path.

du-anomaly - A data item that has been assigned a value becomes undefined without being used.

dd-anomaly - A data item that has been assigned a value is assigned another value without being used in the meantime

Program flow graphs

Describes the program control flow. Each branch is shown as a separate path and loops are shown by arrows looping back to the loop condition node

The control flowgraph is a graphical representation of a program's control structure.

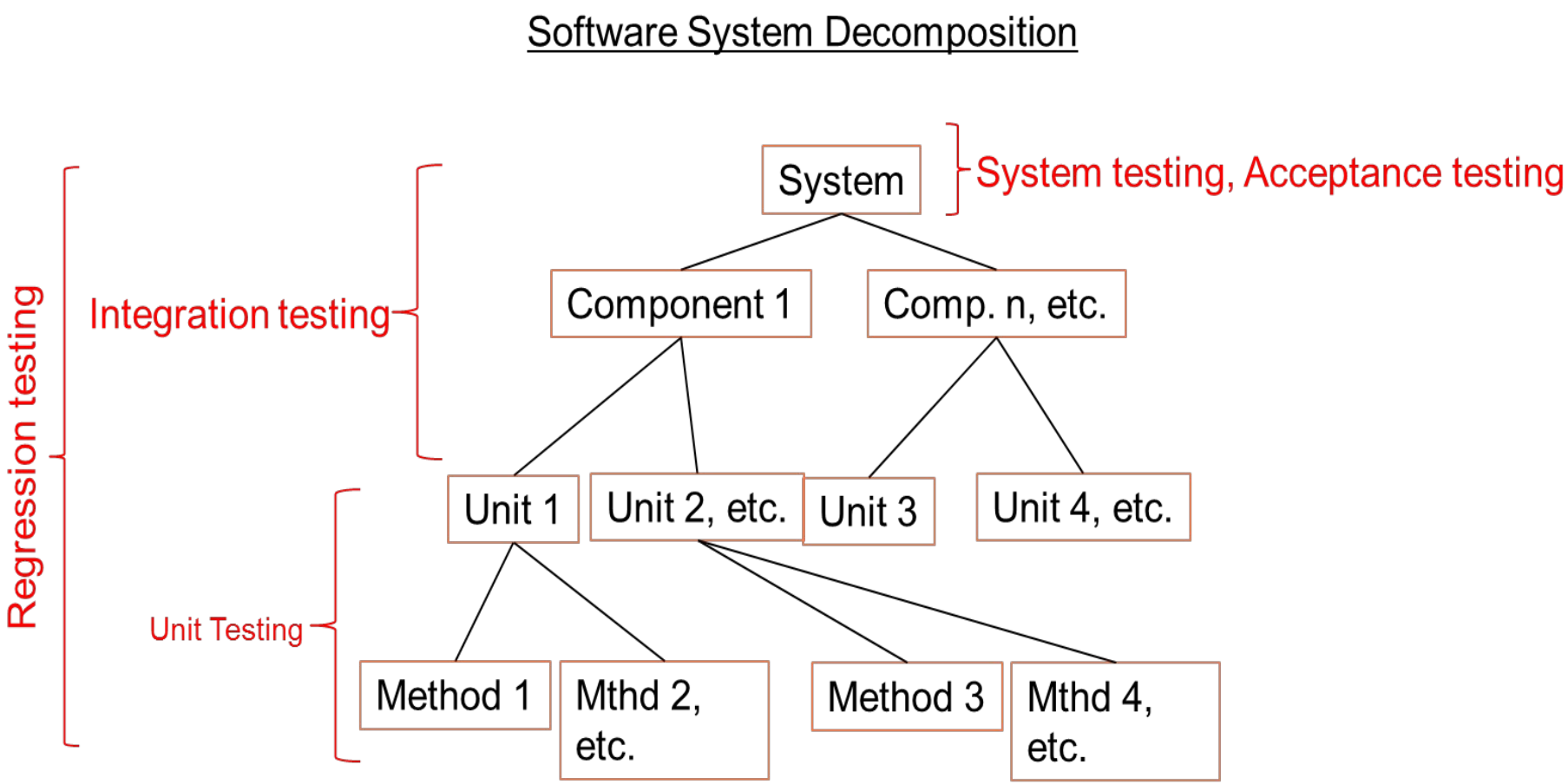
Test design techniques

Test technique	Common features
Specification based Black-Box	Models, either formal or informal, are used for the specification of the problem to be solved, the software or its components. Test cases can then be derived systematically.
Structure based White-box	Information about how the software is constructed is used to derive test cases. The extent of coverage of the software can be measured for existing test cases, and further test cases can be derived systematically to increase coverage.
Experience based	Knowledge of testers, developers, users and other stakeholders about the software, its usage and its environment. They will have knowledge about likely defects and their distribution.

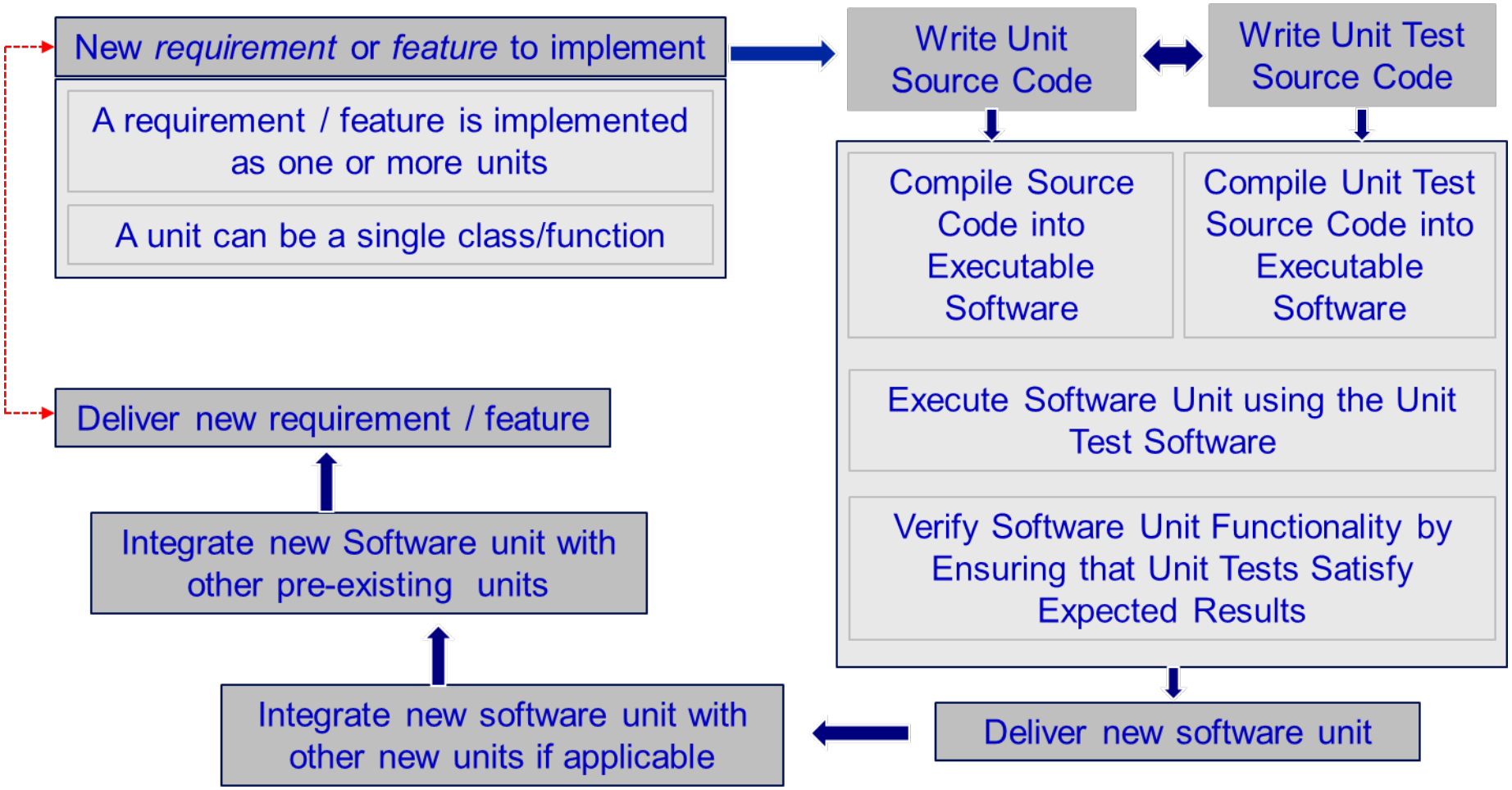
Unit Testing

- Unit Testing
- Pyunit

Where unit testing fits in…



Sequencing of unit testing



What is PyUnit?

- PyUnit is a Python language version of JUnit
- Plays a crucial role in test-driven development
- Advocates the idea of "first testing then coding“ – a good example of Test Driven Development in operation
  - Set up the test data for a piece of code
  - Test the test set up
  - Then write the code
  - Then unit test the code using the test set up
- Unit testing can increase programme productivity
  - If implemented correctly, less time will be required for debugging / resolving defects

Why use unit tests?

- Unit tests help developers to verify that the logic of a piece of code/software is correct.
- Re-performing unit tests (automatically) can identify software regressions introduced by changes in the code.
- Unit tests are therefore very useful for regression testing

About PyUnit

- Used for writing and running unit tests (and test suites)
- Part of the python standard library (*import unittest*)
- Provides assertions for testing expected results
  - E.g. *assertEqual()* to check if an actual value corresponds to the expected value
- Provides test runners for executing tests
  - E.g. a Function called *TestRunner* that can invoke, execute and report on unit test case results (can be used to run suites of tests)

PyUnit – automation and regression

- PyUnit runs automatically (once the tests are written!)
- PyUnit checks results automatically – and flags any errors
- No errors means that all the unit tests have passed as expected
- PyUnit tests can be organised into suites of test cases
- Test suites can be executed any time and much more quickly than manual testing
- This has positive impacts for regression testing and associated software release pace.

Code & Coverage

- Statement testing
- Decision/Branch testing
- Path testing
- Depicting the path from the (pseudo)code
- Coverage

Models and coverage

Statement coverage

- most basic
- **every statement executed at least once**

Branch coverage

- more refined
- **every outcome of every decision executed at least once**

Coverage measurement

Achieving coverage is incremental

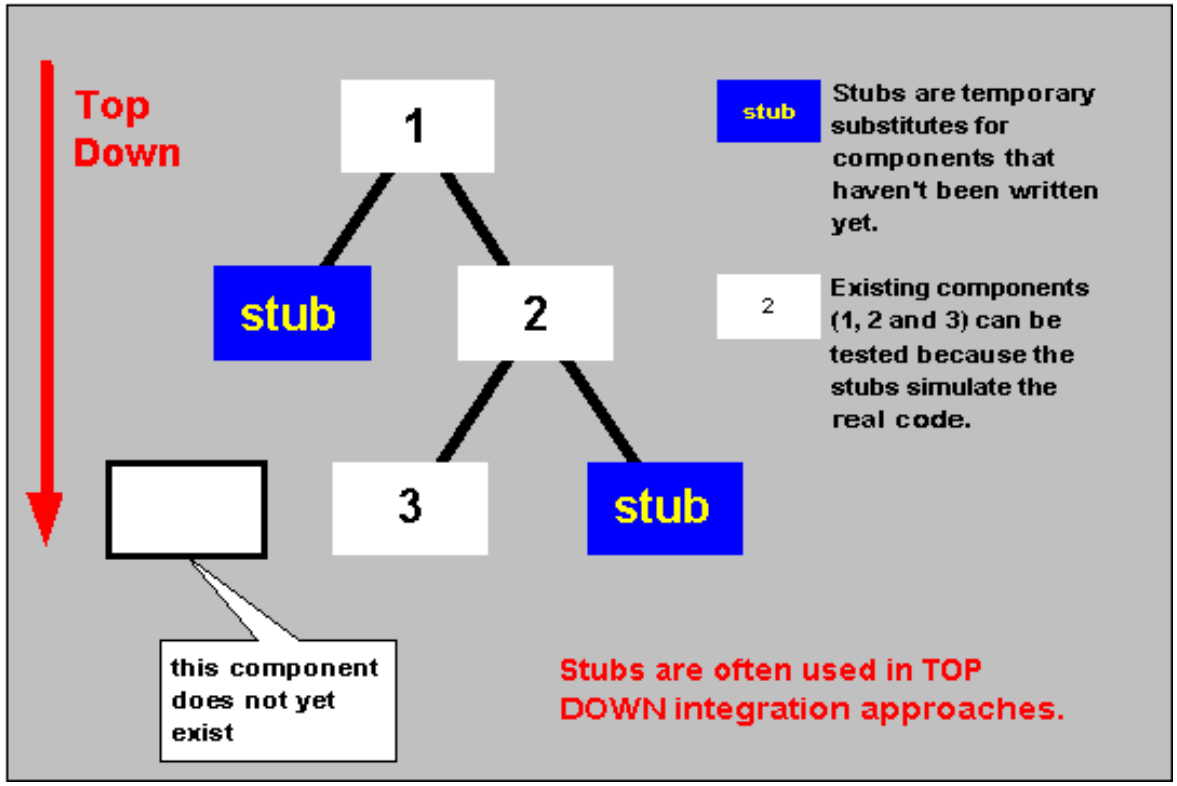
Coverage is normally measured as a percentage.

Statement Coverage = (statements executed / total statements) * 100

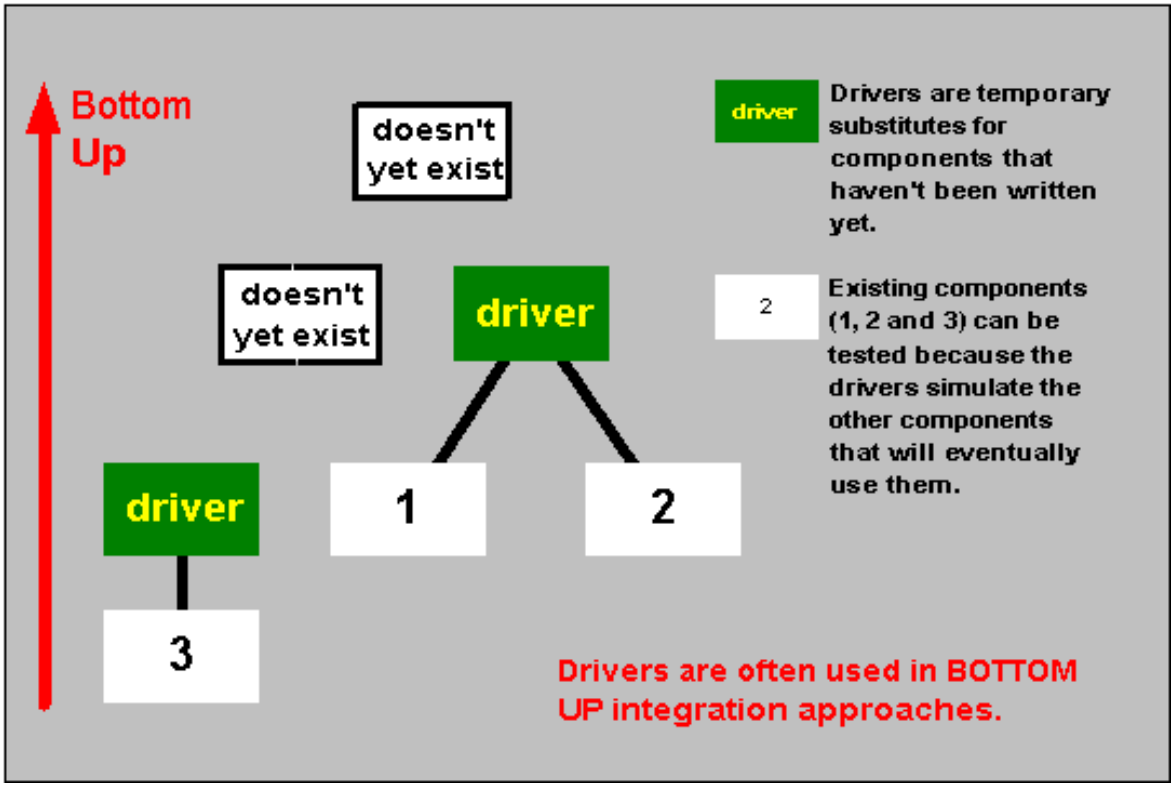
Branch Coverage = (branch outcomes executed / total branch outcomes) * 100

Path Coverage = (paths executed / total number of paths) * 100

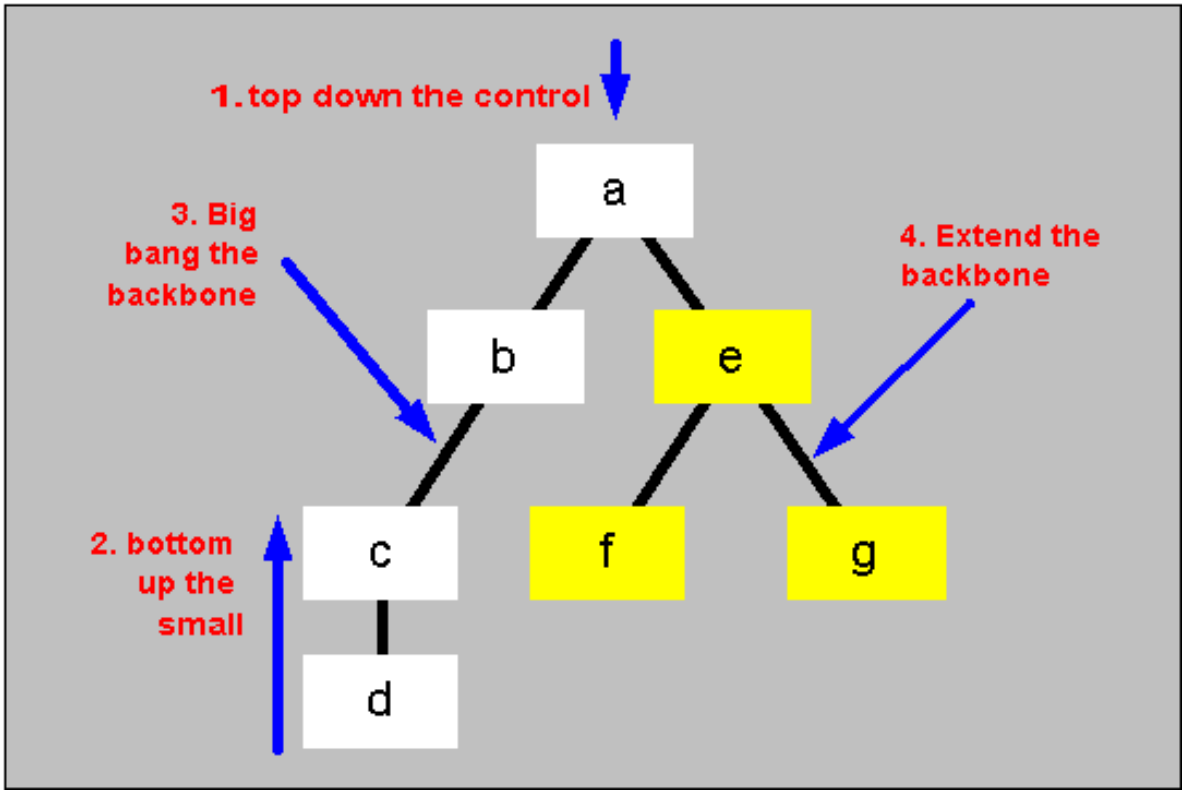
Stubs and top down testing



Drivers and bottom up testing



Mixed integration strategy



Testing Requirements

Importance of requirements

We have seen from earlier material/discussion that inadequate requirements are a commonly reported source of defects.

How can this position be improved?

Poor quality requirement occur:

- partly because requirements elicitation and clarification is challenging;
- but also because some developers (perhaps unwittingly) choose to not invest sufficient time and effort into requirements.

How can requirements be improved?

Testing user stories?

At the heart of the problem…

… we have human and other practical constraints.

We use natural language to express most things.

We assume that:

- We have a sufficiently complete understanding of some topic
- We have communicated our understanding completely to other concerned parties
- Others have completely understood our communication

There are **many gaps in the natural language / human understanding / communication sphere**.

A simple requirement…

Requirement: "*I want to go from Paris to Cannes*".

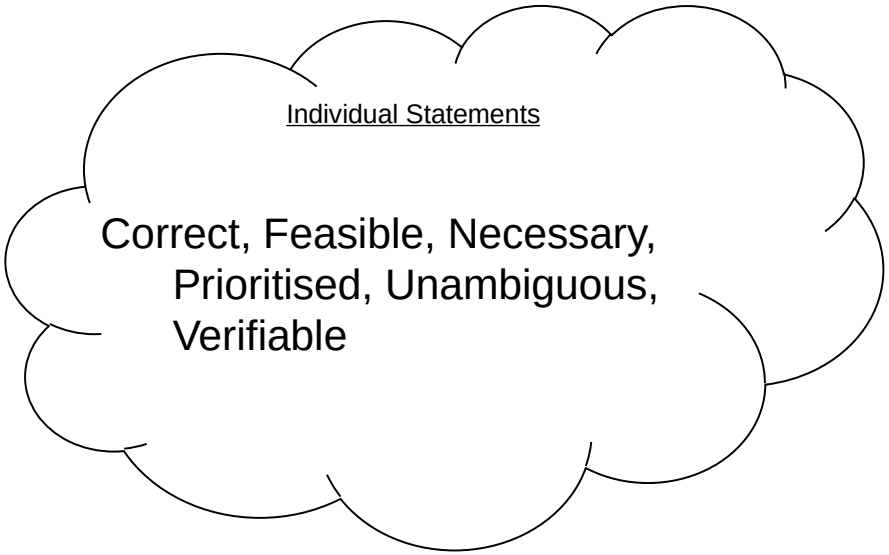
A simple requirement.

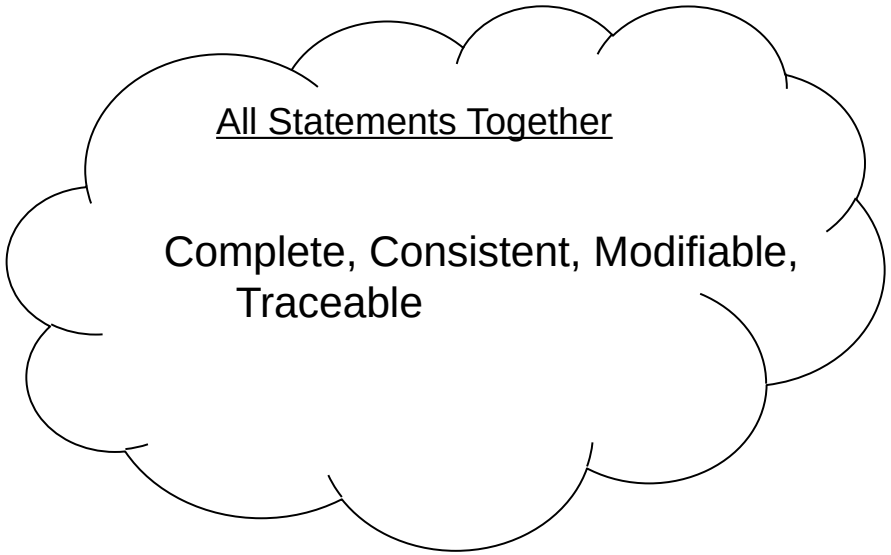
Or so it would seem….

Source: Writing Quality Requirements, Karl E. Wiegers. processimpact.com/articles/qualreqs.html

Quality of Software Requirements

In summary we have: individual-statement and all-statements concerns





Exploratory Testing

Exploratory Testing (ET)

Where the documents that form the basis for test design (e.g. requirements specification(s)) are of very low quality or obsolete or do not exist at all.

Perhaps just the program exists!

This technique may also be applicable when there is a severe time restriction (as it potentially uses far less time than other techniques)

Approach is mainly based on the intuition and experience of the tester

Testing Management

Decision time

When deadline arrives, may have to stop tests now

- Some faults may be acceptable (for this release)
- Some tests may not be run at all

Might have to relax the exit criteria – assess the risk and agree with customer and document the changes

- (release the software now, but testing could continue)

If exit criteria are met, the software/system can be released to the next test phase or production

What if you finish the test plan very early?

- Have you done enough?
- Strengthen the exit criteria, perhaps and create more tests?

Test Estimation

Estimation is not an exact science

It's an approximate calculation or judgement based on:

- direct experience
- related experience
- informed guesswork.

Contingency

Contingency is not a cop-out

You never have complete information, so estimation is imprecise and prone to error

Contingency accounts for potential under-estimation

Some tasks may not require contingency

Agree a percentage contingency on the risky tasks e.g. 10% or 15%

Defect Severity

Severity: the extent to which the defect can affect the software. Can be of following types (sometimes companies use numbers, i.e. 1-5 severity types):

(1) Critical: The defect that results in the termination of the complete system or one or more components of the system and causes extensive corruption of the data. There is no acceptable alternative method to achieve the required results.

(2) Major: The defect that results in the termination of the complete system or one or more components of the system and causes extensive corruption of the data. The failed function is unusable *but there exists an acceptable alternative method* to achieve the required results.

(3) Moderate: The defect that does not result in the termination, but causes the system to produce incorrect, incomplete or inconsistent.

(4) Minor: The defect that does not result in the termination and does not damage the usability of the system and the desired results can be easily obtained by working around the defects.

(5) Cosmetic: The defect that is related to the enhancement of the system where the changes are related to the look and feel of the application.

Defect Priority

Priority: defines the order in which we should resolve a defect. Can be of following types (sometimes companies use numbers, i.e. 1-3 priority types):

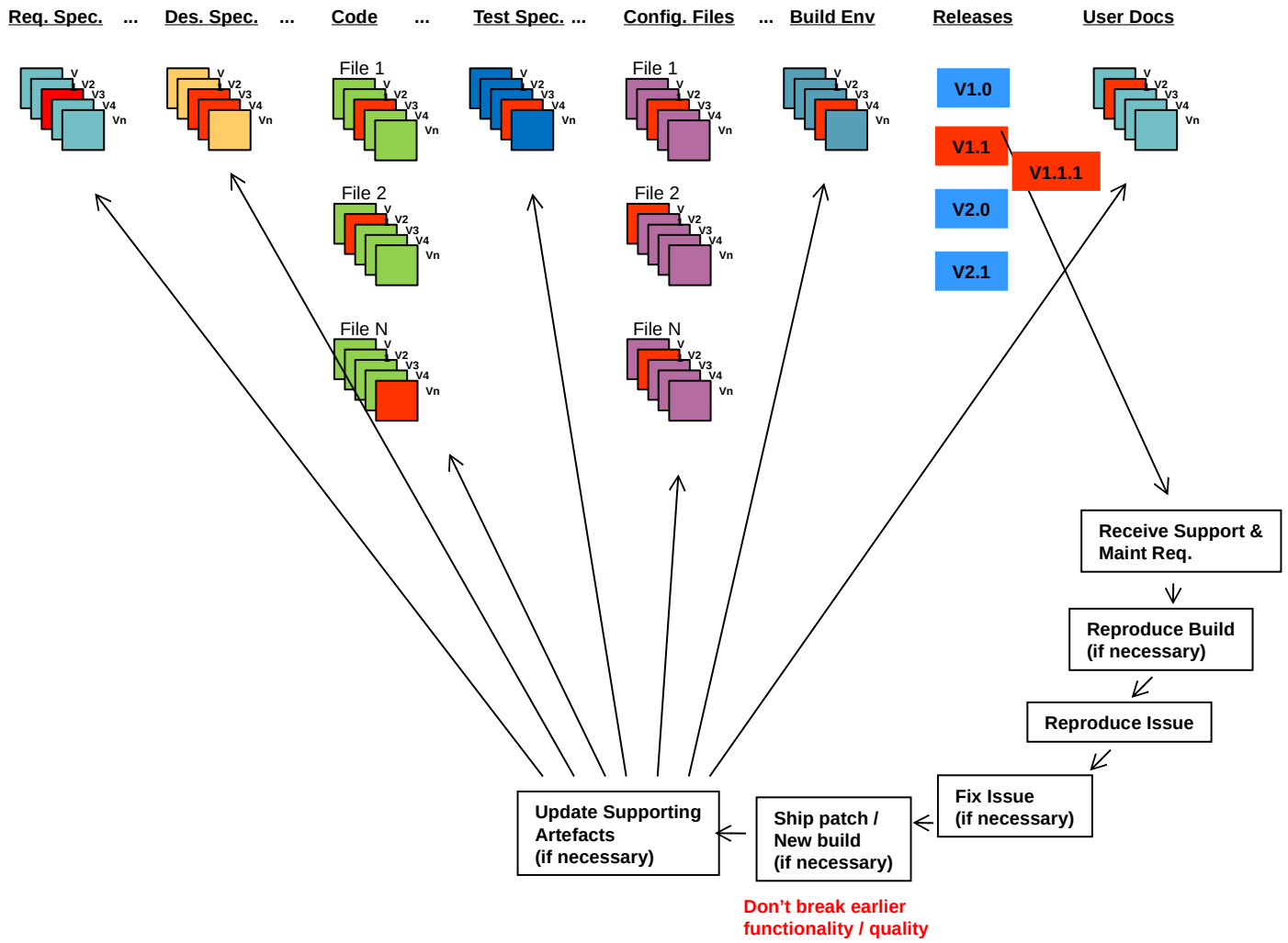
(1) Low: The defect is an irritant which should be repaired, but repair can be deferred until after more serious defect have been fixed.

(2) Medium: The defect should be resolved in the normal course of development activities. It can wait until a new build or version is created.

(3) High: The defect must be resolved as soon as possible because the defect is affecting the application or the product severely. The system cannot be used until the repair has been done.

Sometimes, the priority can be escalated upward based on the reporter.

Configuration Managment



The Tooling Jungle – a very brief overview

Automated UT

pyunit

junit

IDE

IDLE

Eclipse

Most commonly used IDEs?

Defect Tracking

15 Most common defect tracking tools?

Automated Web/GUI/S

CI/Cdep/Cdel

General info

Most commonly used tools?

Top 8 tools?

Note: Some tools do more than just one thing (e.g. defect tracking tools may also be used for project management).

Useful link: http://www.testingtools.com/test-automation/